DAA - Design and Analysis of Algorithms

Session 3 Lab Experiments

Name: Nithya Sree.S

Reg No: 192511058

Question 1: Out of Boundary Paths

Problem: Given an m x n grid and a ball at a starting cell (i, j), find the number of ways to move the ball out of the grid boundary in exactly N steps.

Program (C):

```c
#include <stdio.h>

/* dp[k][r][c] = number of ways to be at cell (r,c) after k steps */
int findPaths(int m, int n, int N, int i, int j) {
    int dp[11][10][10] = {0};
    int dr[4] = {-1, 1, 0, 0};
    int dc[4] = {0, 0, -1, 1};
    int k, r, c, d, count = 0;
    dp[0][i][j] = 1;
    for (k = 0; k < N; k++) {
        for (r = 0; r < m; r++) {
            for (c = 0; c < n; c++) {
                if (dp[k][r][c] == 0)
                    continue;
                for (d = 0; d < 4; d++) {
                    int nr = r + dr[d], nc = c + dc[d];
                    if (nr < 0 || nr >= m || nc < 0 || nc >= n)
                        count += dp[k][r][c];      /* ball went out */
                    else
                        dp[k + 1][nr][nc] += dp[k][r][c];
                }
            }
        }
    }
    return count;
}

int main() {
    printf("Input: m=2,n=2,N=2,i=0,j=0   Output: %d\n", findPaths(2, 2, 2, 0, 0));
    printf("Input: m=1,n=3,N=3,i=0,j=1   Output: %d\n", findPaths(1, 3, 3, 0, 1));
    return 0;
}
```

Output:

```
Output - Program 1 (C)

$ gcc q1.c -o q1 && ./q1
Input: m=2,n=2,N=2,i=0,j=0   Output: 6
Input: m=1,n=3,N=3,i=0,j=1   Output: 12

=== Code Execution Successful ===
```

Complexity: Time: O(N x m x n). Space: O(N x m x n) for the dp table (dynamic programming).

Question 2: House Robber II (Houses in a Circle)

Problem: Houses are arranged in a circle, so the first house is the neighbour of the last one. Adjacent houses have connected security systems, so two adjacent houses cannot be robbed on the same night. Find the maximum money that can be robbed without alerting the police.

Program (C):

```c
#include <stdio.h>

int max(int a, int b) {
    return a > b ? a : b;
}

/* normal house robber on houses from index lo to hi */
int robLine(int nums[], int lo, int hi) {
    int prev = 0, curr = 0, i, temp;
    for (i = lo; i <= hi; i++) {
        temp = max(curr, prev + nums[i]);
        prev = curr;
        curr = temp;
    }
    return curr;
}

/* houses are in a circle: first and last cannot both be robbed */
int rob(int nums[], int n) {
    if (n == 1)
        return nums[0];
    return max(robLine(nums, 0, n - 2), robLine(nums, 1, n - 1));
}

int main() {
    int a[] = {2, 3, 2};
    int b[] = {1, 2, 3, 1};
    printf("Input: nums = [2, 3, 2]\n");
    printf("Output: The maximum money you can rob without alerting the police is %d\n\n", rob(a, 3));
    printf("Input: nums = [1, 2, 3, 1]\n");
    printf("Output: The maximum money you can rob without alerting the police is %d\n", rob(b, 4));
    return 0;
}
```

Output:

Output - Program 2 (C)

```
$ gcc q2.c -o q2 && ./q2
Input: nums = [2, 3, 2]
Output: The maximum money you can rob without alerting the police is 3

Input: nums = [1, 2, 3, 1]
Output: The maximum money you can rob without alerting the police is 4

=== Code Execution Successful ===
```

Complexity: Time: O(n) - the street is scanned twice (without the last house, then without the first house). Space: O(1).

Question 3: Climbing Stairs

Problem: You are climbing a staircase of n steps. Each time you can climb 1 or 2 steps. In how many distinct ways can you climb to the top?

Program (C):

```c
#include <stdio.h>

int climbStairs(int n) {
    int dp[50], i;
    dp[0] = 1;
    dp[1] = 1;
    for (i = 2; i <= n; i++)
        dp[i] = dp[i - 1] + dp[i - 2];
    return dp[n];
}

int main() {
    printf("Input: n=4   Output: %d\n", climbStairs(4));
    printf("Input: n=3   Output: %d\n", climbStairs(3));
    return 0;
}
```

Output:

Output - Program 3 (C)

```
$ gcc q3.c -o q3 && ./q3
Input: n=4   Output: 5
Input: n=3   Output: 3

=== Code Execution Successful ===
```

Complexity: Time: O(n). Space: O(n) for the dp array.

Question 4: Unique Paths in a Grid

Problem: A robot is at the top-left corner of an m x n grid and can only move down or right. How many possible unique paths are there to reach the bottom-right corner?

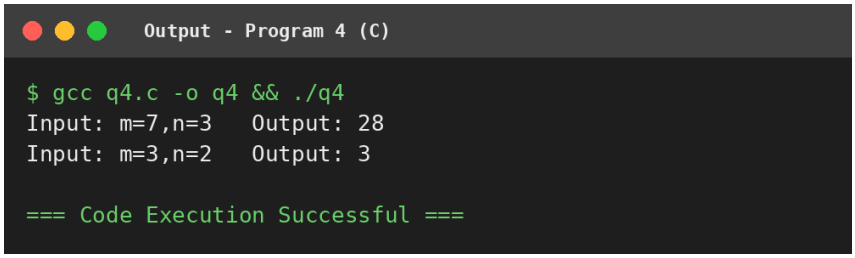
Program (C):

```c
#include <stdio.h>

int uniquePaths(int m, int n) {
    int dp[20][20], i, j;
    for (i = 0; i < m; i++)
        dp[i][0] = 1;
    for (j = 0; j < n; j++)
        dp[0][j] = 1;
    for (i = 1; i < m; i++) {
        for (j = 1; j < n; j++)
            dp[i][j] = dp[i - 1][j] + dp[i][j - 1];
    }
    return dp[m - 1][n - 1];
}

int main() {
    printf("Input: m=7,n=3   Output: %d\n", uniquePaths(7, 3));
    printf("Input: m=3,n=2   Output: %d\n", uniquePaths(3, 2));
    return 0;
}
```

Output:



Complexity: Time: O(m x n). Space: O(m x n) for the dp table.